

CS 4341 Assignment 3

Learn to Play Tag

Due October 4, 2026

1 Overview

Design a reinforcement-learning agent that learns to play tag through self-play. Choose a discrete state representation, a reward function, an exploration policy, and a learning algorithm. Implement the supplied abstract agent interface in `main.py`. The framework runs the games and delivers experience to your agent. Your grade is based on automated compatibility tests (20%) and a five-minute presentation explaining your implementation (80%). Tournament results recognize performance and leave your assignment grade unchanged.

2 Tag

Players occupy distinct cells on a square grid. One player is *It*. Turns follow the fixed order in the state. A player may move one cell north, south, west, or east when the destination is on the board and unoccupied. A player who is *It* may instead tag an adjacent player, transferring *It* while both positions stay fixed. The newly tagged player must move or wait before tagging; this prevents an immediate tag back. The starting *It* player also moves or waits before its first tag.

A player who is *It* may always wait. Other players may wait only when every move is blocked. Finishing a player's own turn as *It* increases that player's `it_turns` by one. A successful tag passes *It* to another player and increases the acting player's `tags` by one. Games end at a fixed turn budget. These statistics are available for your reward design. Use `tag.py` as the authority for legal actions and transitions.

3 Getting Started

Extract the whole starter bundle into one directory and follow `README.md` to set up Python 3.10 or newer and install the test dependency. That guide includes the file list and commands for testing and local simulation.

Edit `main.py`. Keep your implementation in that file and use Python's standard library. Store your learned model and hyperparameters on `self`. Set `GROUP_NAME` to your group's nonempty tournament name. Preserve the class name `Agent`, its inheritance from `BaseAgent`, and the supplied method signatures.

4 Agent Interface

The supplied `BaseAgent` in `reinforcement_learning.py` defines five abstract methods. Complete them in `Agent`, along with its constructor. The encoder and reward function receive complete boards as `TagState` objects. Action selection and model updates receive integer state encodings, together with the legal actions and terminal flags those methods need.

The `TagState` provides `n_rows`, `n_cols`, `players`, aligned `positions` and `scores`, `next_player`, `tagged_player`, and `just_tagged`. Its turn counts completed player turns, and `max_turns` is the episode's limit. Use `position_of(player)` and `score_of(player)` for player-specific information.

Board inspection and calls to `Tag` methods belong in `encode_state` and `calculate_reward`; retain state history as encoded integers.

`__init__(self, seed)`

Initialize your model and hyperparameters. Keep `super().__init__(seed)` and use the resulting `self.rng` for reproducible randomness. The framework constructs a fresh instance for each independent training run and reuses that instance and its in-memory model for evaluation.

`init_episode(self) -> EpisodeConfig`

Choose the next training episode's square board size, number of opponents, and maximum number of steps:

```
return EpisodeConfig(side_length=4, n_opponents=3, max_steps=100)
```

All three fields must be positive integers. `max_steps` counts total player turns across the whole episode; each move, tag, or wait uses one step. The total number of players is `n_opponents + 1`. The configuration raises `ValueError` when that total exceeds `side_length ** 2`. A 4-by-4 grid therefore supports at most 15 opponents. You may vary the configuration across episodes.

The framework calls this method before the first training episode and before each later episode while update budget remains. Set up episode-specific state here, such as exploration schedules or per-player history, and preserve learned model data across episodes. Evaluation uses the trained instance and encodes each new observation before selecting an action.

`encode_state`

```
encode_state(self, state: TagState, player: Player) -> int
```

Convert the board into one Python integer from `player`'s perspective. The result must satisfy `type(result) is int`. Make this method a pure conversion: the same board and player produce the same integer without changing agent state.

Choose which information to retain or aggregate: positions, relative distances, local neighborhoods, the *It* role, tag immunity, board size, score statistics, or remaining time. Pack the features into an integer that captures what your policy and learning algorithm need. Explain the tradeoff between the information preserved and the experience needed to learn over your state space.

`calculate_reward`

```
calculate_reward(self, state: TagState, action: Action,
                 next_state: TagState, player: Player,
                 terminal: bool) -> float
```

Return a finite Python `float` using the complete board states before and after the transition. You choose the components and their scales. Successful tags, time spent *It*, distance, escape room, and terminal performance are possible signals.

A training transition spans `player`'s chosen action and the intervening players' actions. The `state` is the board immediately before that player's action; `next_state` is the board at that player's next turn or at

game end. Evaluate both boards for the supplied **player**. The flag **terminal=True** identifies a finished episode.

select_action

```
select_action(self, state: int,
               legal_actions: tuple[Action, ...],
               training: bool) -> Action | None
```

The integer **state** represents the current player's perspective. Return one of the supplied **legal_actions**. An empty tuple means the episode is terminal; return **None** in that case. The Boolean flag selects the policy's mode:

- **training=True**: apply your exploration policy during self-play;
- **training=False**: use your learned evaluation policy during simulations and the tournament.

Choose evaluation randomization only when it is an intentional part of your final policy. Keep learned model updates in **update_model**.

update_model

```
update_model(self, state: int, action: Action, reward: float,
              next_state: int, next_actions: tuple[Action, ...],
              player: Player, terminal: bool) -> None
```

Update the learned model on **self** and return **None**. The framework supplies the integer encodings of both boards and the reward returned by **calculate_reward** for that transition. Both integers use the supplied **player's** perspective. Use your chosen reinforcement-learning update rule. The tuple **next_actions** contains the legal actions at that player's next turn. When **terminal=True**, this tuple is empty and future action value is zero.

Keep any player-specific action history or traces indexed by **player**. Account for updates from other players between this player's decisions.

5 Supplied Training Loop

The framework runs self-play through **train_agent(agent, seed, updates)**. One shared agent controls every player, so all opponents use your current policy and learned model:

1. Call **init_episode()** and construct the requested square board, with seeded starting positions and a seeded starting *It* player.
2. Encode the board for the current player, call **select_action** with that integer, the legal actions, and **training=True**, then apply its action.
3. Before a player acts again, encode the new board from that player's perspective. Pass the complete boards from its preceding decision interval to **calculate_reward**, then pass the reward and integer transition to **update_model**.
4. End the episode after **max_steps** total player turns, including when the limit falls partway through a round. Deliver pending transitions with the terminal state, subject to the remaining update budget, then begin the next episode with **init_episode()** if budget remains.

Full training provides **exactly 1,000,000 calls to `update_model` in total**, counted across all players and episodes. The loop stops immediately at the budget, even partway through a game or its final terminal updates. Some selected actions may therefore have pending transitions that are never delivered. Use the supplied loop for environment interaction and place your learning update inside the callback.

The framework’s `encode_observation` helper calls your encoder and validates its integer result. Its `choose_action` helper encodes an observation, passes the integer and legal actions to your policy, and validates the chosen action. The supplied trainer and evaluation runners use these helpers.

Training run	Total updates	Time limit
Public test	1,000	30 seconds
Private grading	10,000	30 seconds
Tournament	1,000,000	1 hour per agent

The constructor and direct method checks must finish within one second. Each evaluation decision, including encoding and action selection, must take less than one second. Implement updates efficiently enough to complete the full training budget.

6 Testing and Local Simulation

Use the commands in `README.md` to run the public tests and local simulation. The tests accept different state abstractions, reward scales, and learning algorithms. They check interface compatibility and completion; your presentation explains how the implementation learns. Make your policy work across square board sizes and player counts, including configurations different from your training choices.

7 Tournament

Every submission that passes all automated tests enters the same multiplayer arena. With N entrants, the tournament runs N seeded games of 100 rounds each, and each entrant starts *It* exactly once. One round gives every active player one turn. The square board grows with the number of entrants. Player order rotates between games, and the starting *It* player follows a reproducibly shuffled entrant order.

Each submission receives its full self-play training run before evaluation. The runner retains that instance for all N games, encodes each observation, and calls `select_action` with `training=False`.

An illegal or malformed action, exception, timeout, or process failure forfeits the current game. The runner logs the failure and removes that player, freeing its cell. If the removed player is *It*, the next active player in turn order becomes *It* and must move or wait before tagging. Remaining rounds skip removed players; a game ends early when fewer than two players remain. Every entrant returns at the start of each later game, including its game starting *It*.

Rank all players by fewest total forfeits, then fewest total `it_turns`, then most total successful `tags`, summed across all games. Players tied on all three totals share a rank; players tied for first share the championship. Seeded order determines the display order among tied players.

8 Submission and Presentation

Upload two separate files to Canvas: your completed `main.py` and `presentation.pdf`. Export one slide per page, in presentation order. You may create the slides in any tool.

Prepare a five-minute presentation that matches your submitted code. Use the rubric below to structure your explanation.

Visual-Only Presentation Rule

Communicate your explanation aloud and through visuals. Limit displayed words to concise slide titles, section headers, and labels that identify elements within figures, diagrams, plots, or tables. Title slides may identify the presentation, presenters, course, and date.

- **Titles and labels:** use topic titles such as “State Encoding,” axis labels, legends, diagram-node labels, table row/column labels, and short category names. Labels name visual elements using concise terms. Numbers and mathematical notation may appear within these visuals.
- **Spoken explanation:** convey your reasoning, findings, and conclusions aloud. This division between visual labels and spoken explanation applies equally to slides, tables, diagrams, images, screenshots, animations, and videos.
- **Model explanations:** use labeled state diagrams, learning curves, board illustrations, and action-value plots or tables. Keep source code, pseudocode listings, and terminal output in your development workspace.

The rule applies to the entire submitted presentation PDF, including appendix slides, and all materials displayed during the presentation. Keep speaker notes in your private preparation materials and export only the slides to PDF.

9 Grading

Automated Compatibility Tests: 20%

Each of the 20 tests is worth 1%:

- top-level `main.py` and group name: 2 tests;
- `init_episode` and valid episode configuration: 1 test;
- deterministic integer state encoding on four states: 4 tests;
- finite rewards on four transitions: 4 tests;
- `update_model` on regular and terminal transitions: 1 test;
- completion of the supplied self-play training loop: 1 test;
- timely legal evaluation actions on five states: 5 tests;
- terminal-state handling: 1 test; and
- completion of a seeded multiplayer game: 1 test.

Gradescope uses different seeded states and transitions for its private checks.

Presentation: 80%

Time limit: 20%. Finish within 5 minutes for 20%; finish after 5 minutes and within 7 minutes for 10%; exceed 7 minutes for 0%.

Visual-only rule: 20%. Full compliance with the visual-only presentation rule earns 20%. Any prohibited text earns 0% for this entire criterion. Timing and technical content are graded separately.

Technical content: 40%.

- **Discrete states: 10%.** Explain the representation, retained and discarded information, and consequences for learning.
- **Reward: 10%.** Explain components, scales, strategic intent, and important tradeoffs.
- **Reinforcement-learning algorithm: 20%.** Correctly explain learning and action selection, including model updates, self-play, episode setup, exploration and exploitation, hyperparameters, learned representation, and tournament use.